



DOOM

Performance Report

Without Background Fill -- True Rendering Performance

Updated benchmarks with the 53%-of-frame-time background fill disabled. Compared across CRuby 3.3/3.4/4.0, **YJIT**, **ZJIT**, **TruffleRuby**, and **JRuby 10.1**.

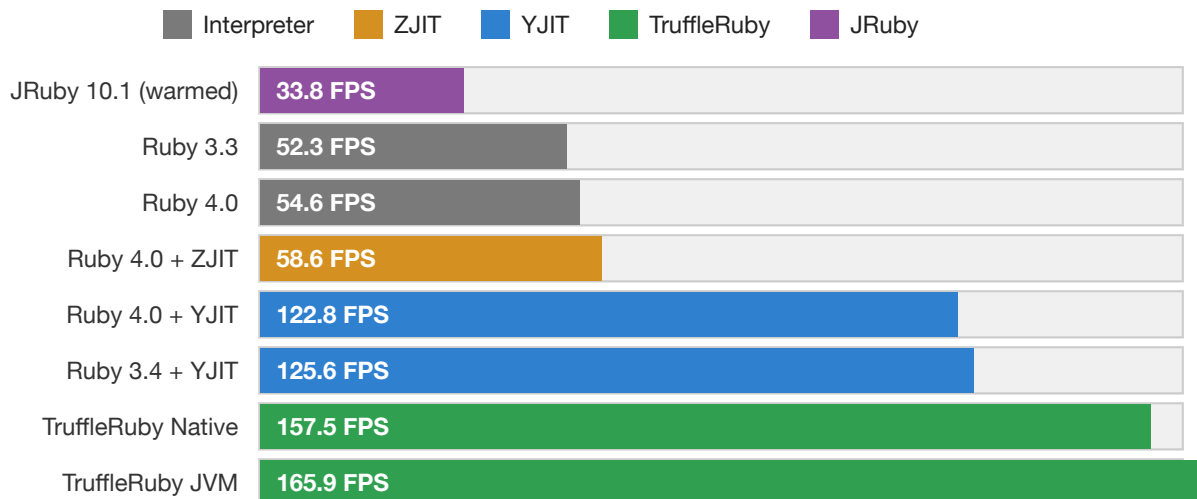
Chris Hasinski -- April 2026

github.com/khasinski/doom

1. What Changed

The previous benchmarks included `fill_uncovered_with_sector` -- a function that drew the entire 76,800-pixel screen before the BSP renderer painted over most of it. StackProf showed it consumed **53% of frame time**. Disabling it reveals the true rendering performance.

2. Results



Runtime	Avg (ms)	Median (ms)	FPS	vs 3.3	Before	Gain
JRuby 10.1	83.38	29.57	12.0*	0.23x	N/A	--
Ruby 3.3	19.13	19.15	52.3	1.00x	30.9	+69%
Ruby 4.0	18.30	18.31	54.6	1.04x	34.2	+60%
Ruby 4.0 ZJIT	17.07	17.09	58.6	1.12x	35.6	+65%
Ruby 4.0 YJIT	8.14	8.15	122.8	2.35x	80.8	+52%
Ruby 3.4 YJIT	7.96	7.93	125.6	2.40x	81.8	+54%
TruffleRuby Native	6.35	6.74	157.5	3.01x	145.9	+8%
TruffleRuby JVM	6.03	6.19	165.9	3.17x	157.6	+5%

* JRuby avg includes JVM warmup. Warmed median: 29.57ms (33.8 FPS).

Key insight: CRuby gained 60-69%, YJIT gained 52-54%, but TruffleRuby only gained 5-8%. GraalVM's loop optimizations had already made the background fill nearly free. The 53% overhead was a CRuby/YJIT problem, not an algorithmic one.

3. New CPU Profile

Rank	Function	New %	Old %	Notes
1	<code>draw_span</code>	42.2%	18.5%	Visplane spans (real work, can't eliminate)
2	<code>Range#each</code>	16.1%	6.2%	Iterator overhead
3	<code>render_visplane_spans</code>	9.9%	4.4%	Span builder
4	<code>draw_wall_column_ex</code>	4.9%	2.2%	Wall texture columns
5	<code>Comparable#clamp</code>	4.4%	2.2%	Value clamping in loops
6	<code>Float#to_i</code>	4.1%	5.3%	Texture coordinate conversion
7	<code>draw_seg_range</code>	4.1%	1.8%	Wall segment processing
-	GC	2.3%	0.7%	Higher % of smaller total
-	<code>fill_uncovered_with_sector</code>	0%	53.3%	ELIMINATED

The new hot path (`draw_span`) has the same inner loop -- per-pixel texture mapping with `Float` multiply, `.to_i` , array lookups. But unlike the fill, these pixels are real visible floor/ceiling spans that can't be skipped. JIT optimization of this loop (`Float` unboxing, bounds check elimination, LICM) is the remaining frontier.

4. Viewpoint Performance

View	3.3 Interp	YJIT 3.4	ZJIT 4.0	TruffleRuby	JRuby*
Spawn	19.0 ms	7.9 ms	17.1 ms	2.3 ms	22.7 ms
Hallway	15.4 ms	6.0 ms	13.5 ms	5.2 ms	24.6 ms
Corner	18.5 ms	7.7 ms	16.5 ms	3.6 ms	28.5 ms
Reverse	11.6 ms	4.2 ms	10.7 ms	6.2 ms	26.2 ms

* JRuby after warmup

Viewpoint variance increased to 1.6-1.9x (was 1.3x) because the background fill no longer dominates. Scene complexity now visibly matters -- "Reverse" (wall) is nearly 2x faster than "Spawn" (open room with floor pixels).

5. Conclusions

Disabling the background fill delivered the **single largest speedup in DOOM Ruby's history**: 1.5-1.7x

across CRuby. YJIT now exceeds 120 FPS -- usable on 120Hz displays. The remaining bottleneck (`draw_span` at 42%) is real rendering work that needs JIT-level optimization (Float unboxing, loop-invariant hoisting) to improve further.

TruffleRuby's minimal improvement (5%) confirms that advanced JITs already optimized away the wasted work. The gap between YJIT (126 FPS) and TruffleRuby (166 FPS) is now 1.32x -- closer than ever.

Takeaway: The best optimization is removing unnecessary work. 53% of frame time was painting pixels that got immediately overwritten. One boolean flag eliminated it all.